

GitHub y organización del proyecto

Table of contents

1	1. Objetivo de este documento	1
2	2. Organización de GitHub por proyecto	2
3	3. Repositorios mínimos recomendados	2
4	4. Projects, Milestones e Issues	3
4.1	4.1. Milestones (hitos)	3
4.2	4.2. Issues (tareas)	4
4.3	4.3. Project (tablero Kanban)	4
5	5. Ramificación (branches) y flujo de trabajo básico	5
6	6. Integración continua (CI): recomendada, no obligatoria	5
6.1	6.1. ¿Cuándo tiene sentido usar CI?	5
6.2	6.2. Ejemplo de workflow mínimo (Python)	6
7	7. Checklist inicial para el alumno	7

1 1. Objetivo de este documento

Este documento explica **cómo organizaremos el proyecto en GitHub**:

- Creación de una **organización por proyecto** (en lugar de usar tu cuenta personal).
- Estructura recomendada de **repositorios**.
- Uso de **Projects, Issues y Milestones** para gestionar el trabajo.
- Recomendaciones generales sobre **integración continua (CI)**.

La CI se propone como opción recomendada, no obligatoria: tú decides si la aplicas en tu TFG/TFM.

En este handbook sí se usa CI para generar la web, y puedes tomarlo como ejemplo.

2 2. Organización de GitHub por proyecto

En la mayoría de TFG/TFM que dirija seguiremos esta idea:

- Crear una **organización específica en GitHub** para el proyecto.
- El nombre será un **nombre en clave**, no tu nombre personal.

Ejemplos de nombres de organización:

- TempusUGR
- PastilleroUGR
- HorarioUGR
- SenseUGR

Ventajas de usar organizaciones:

- Puedes agrupar varios repositorios (código, documentación, infraestructura...).
 - Es más fácil colaborar con otras personas (añadir miembros con distintos permisos).
 - El resultado final queda más limpio y “profesional” que un repo suelto en una cuenta personal.
-

3 3. Repositorios mínimos recomendados

Dentro de la organización de tu proyecto, lo normal será tener al menos:

1. Repositorio de código principal

Ejemplos:

- `pastillero-app`
- `tempus-frontend`, `tempus-backend`

Estructura típica:

```
src/          # Código fuente
tests/        # Tests automatizados (si aplica)
docs/         # Documentación específica de ese repo (si la hay)
.github/
  workflows/  # Workflows de CI (opcional)
README.md
```

2. Repositorio de documentación/memoria

Por ejemplo:

- pastillero-docs
- tempus-docs

Aquí puedes guardar:

- Memoria del TFG/TFM (en LaTeX, Quarto, etc.).
- Anexos.
- Diagramas, figuras, datos, etc.

En proyectos más grandes se pueden añadir más repositorios (microservicios, librerías comunes, etc.), pero para un TFG/TFM normal con estos dos suele ser suficiente.

4 4. Projects, Milestones e Issues

4.1 4.1. Milestones (hitos)

Usa **milestones** para agrupar issues según fases del TFG/TFM.

Ejemplo de milestones:

- M1 - Análisis y planificación
- M2 - Diseño / arquitectura
- M3 - Implementación MVP
- M4 - Evaluación
- M5 - Cierre y memoria

Cada milestone debe tener:

- Una **fecha objetivo** razonable.
- Un conjunto de issues asignados.

4.2 4.2. Issues (tareas)

Cada tarea importante del proyecto debe ser un **issue**:

- Usa títulos claros, por ejemplo:
 - Definir historias de usuario iniciales
 - Diseñar modelo de datos
 - Implementar pantalla de toma de medicación
 - Escribir sección 3. Metodología de la memoria
- Añade una descripción breve:
 - Qué se espera conseguir.
 - Criterios de aceptación (cuando tenga sentido).
- Etiquetas recomendadas:
 - backend, frontend, documentation, research, ui-ux, bug, enhancement, etc.
- Asigna el issue a:
 - Un milestone.
 - Opcionalmente, a ti mismo (siempre serás el responsable principal).

4.3 4.3. Project (tablero Kanban)

Crea un **Project** de GitHub (board) asociado a la organización o a un repo principal.

Configuración mínima:

- Columnas:
 - Backlog
 - En curso
 - En revisión
 - Hecho

Cada issue se añade al Project y se mueve de columna según su estado.

Esto permite ver de un vistazo:

- Qué tareas hay pendientes.
- Qué estás haciendo ahora.
- Qué está ya terminado.

5 5. Ramificación (branches) y flujo de trabajo básico

No vamos a imponer un flujo Git muy complejo, pero sí se recomiendan unas pautas mínimas:

- Mantener una rama principal (`main` o `master`) **siempre en estado estable**.
 - Para cambios medianos/grandes:
 - Crear una rama de feature, por ejemplo:
 - * `feature/pantalla-tomas`
 - * `feature/modelo-datos`
 - Hacer commits en esa rama.
 - Crear un **pull request (PR)** a `main` cuando esté listo.
 - Si trabajas solo, puedes auto-mergear el PR, pero:
 - El PR sirve como “paquete” de cambios.
 - Puedes usar la descripción del PR para documentar qué has hecho.
-

6 6. Integración continua (CI): recomendada, no obligatoria

6.1 6.1. ¿Cuándo tiene sentido usar CI?

Tiene sentido plantearse CI cuando:

- Tienes una batería de **tests automatizados** (unitarios, de integración, etc.).
- El proyecto va a crecer o quieres mantener cierta disciplina técnica.
- Quieres automatizar tareas repetitivas:
 - Ejecutar tests en cada push.
 - Generar informes.
 - (Opcional) Generar builds o documentación.

No es obligatorio para el TFG/TFM, pero:

- Si lo usas bien, **suma puntos** a nivel de calidad y profesionalidad.
- Puedes tomar como referencia el workflow que usamos en este handbook (`quarto-handbook.yml`).

6.2 6.2. Ejemplo de workflow mínimo (Python)

Este es solo un ejemplo genérico para proyectos en Python:

```
# .github/workflows/tests.yml
name: Run tests

on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          pip install -r requirements.txt

      - name: Run tests
        run: |
          pytest
```

Adáptalo a:

- Node/TypeScript, Java, etc., según el stack de tu proyecto.
- Solo si realmente vas a usar tests.

7 7. Checklist inicial para el alumno

Cuando empieces tu TFG/TFM, deberías:

1. Crear la **organización** de GitHub con un nombre en clave del proyecto.
2. Crear al menos:
 - ☐ Un repositorio para el código.
 - ☐ Un repositorio para la documentación/memoria.
3. Configurar un **Project** (tablero) y los primeros **milestones**.
4. Crear los primeros issues de backlog (a partir del documento de trabajo y de las conversaciones con el director).
5. Documentar en el README del repositorio principal:
 - Objetivo del proyecto.
 - Cómo ejecutar el código.
 - Tecnologías principales.

La integración continua (CI) es recomendable pero opcional:

[] Si decides usarla, crea un workflow sencillo para lanzar tests o validaciones básicas en cada push.